Unit Testing Using PyUnit

Code Examples

Some coding examples

1. Sample code to add two numbers and some associated unit test code (using assertEqual() and assertNotEqual())
2. Sample code with a broken unit test – to examine the type of output we receive
3. Sample code to check if a number is a prime number and associated unit test code (using assertTrue() and assertFalse())
4. Sample code to run the tests in (1) and (2) above as a test suite

We're using **python 3.7** and the associated IDLE IDE.

(1) Python code: add.py

```
def add(x,y):
        return x+y
```

Obviously, from a practical perspective, do not need to write code to test that python can add two numbers reliably

This sample code is created for the purpose of demonstrating unit testing in action

This file will be stored as add.py and in a subdirectory called ‘code’

(1) Python code: test_add.py

```
import unittest
from code.add import add

class AddTestCase(unittest.TestCase):

    def test_one(self):
        self.assertEqual(add(1,1),2)

    def test_two(self):
        self.assertEqual(add(1,2),3)

    def test_three(self):
        self.assertNotEqual(add(1,3),5)

if __name__ == '__main__':
    unittest.main()
```

Unittest is the python package for pyunit

Import ‘add’ from the add package in the subdirectory ‘code’

Create a new test case class to test ‘add’

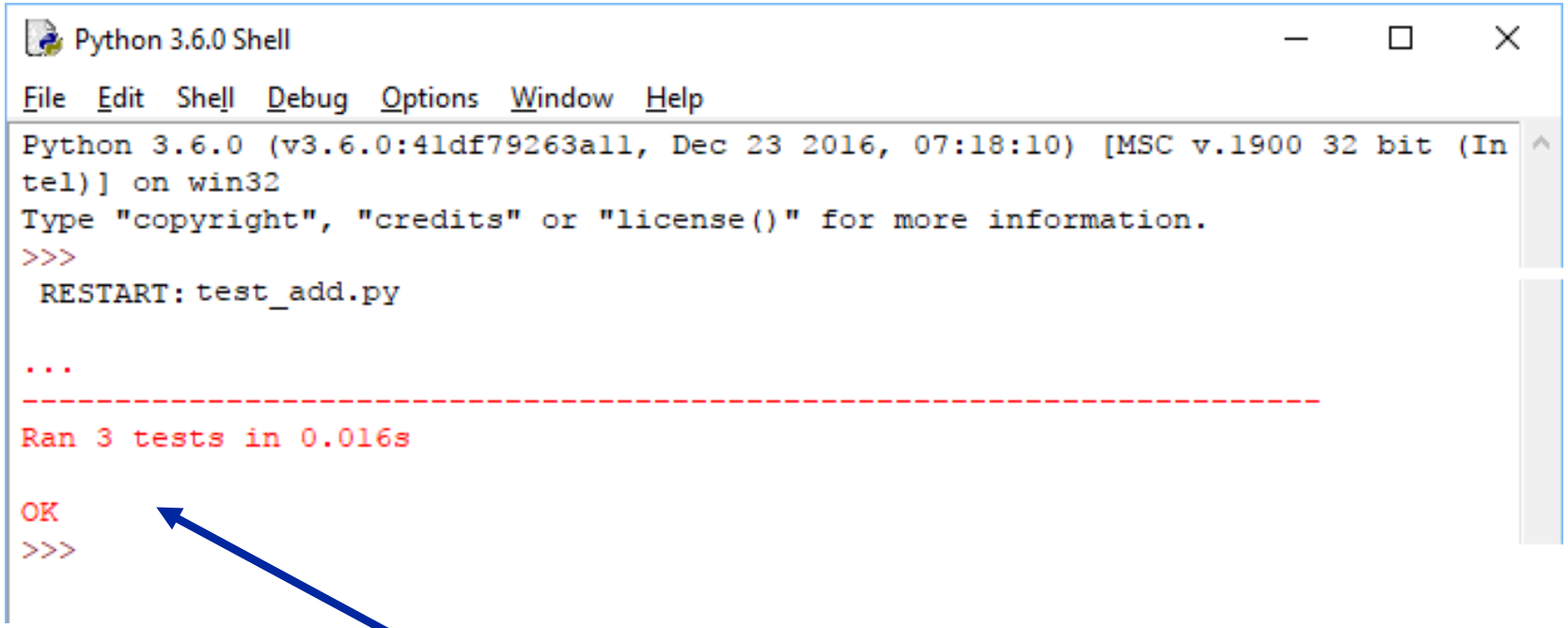
Positive test case: Check if the return value from add(1,1) is 2

Positive test case: Check if the return value from add(1,2) is 3

Negative test case: Check if the return value from add(1,2) is not equal to 5

Allows us to run our tests as if it was the main line of the program (i.e. the place the program starts)

(1) What happens when we run this code?



All three tests ran, everything is “ok”

In IDLE, you can use the F5 shortcut key to automatically run the code.

(2) When a test fails… (because of a faulty test case)

```
import unittest
from code.add import add

class AddTestCase(unittest.TestCase):

    def test_one(self):
        self.assertNotEqual(add(1,1),2)

    def test_two(self):
        self.assertEqual(add(1,2),3)

    def test_three(self):
        self.assertNotEqual(add(1,3),5)

if __name__ == '__main__':
    unittest.main()
```

Unittest is the python package for pyunit

Import ‘add’ from the add package in the subdirectory ‘code’

Create a new test case class to test ‘add’

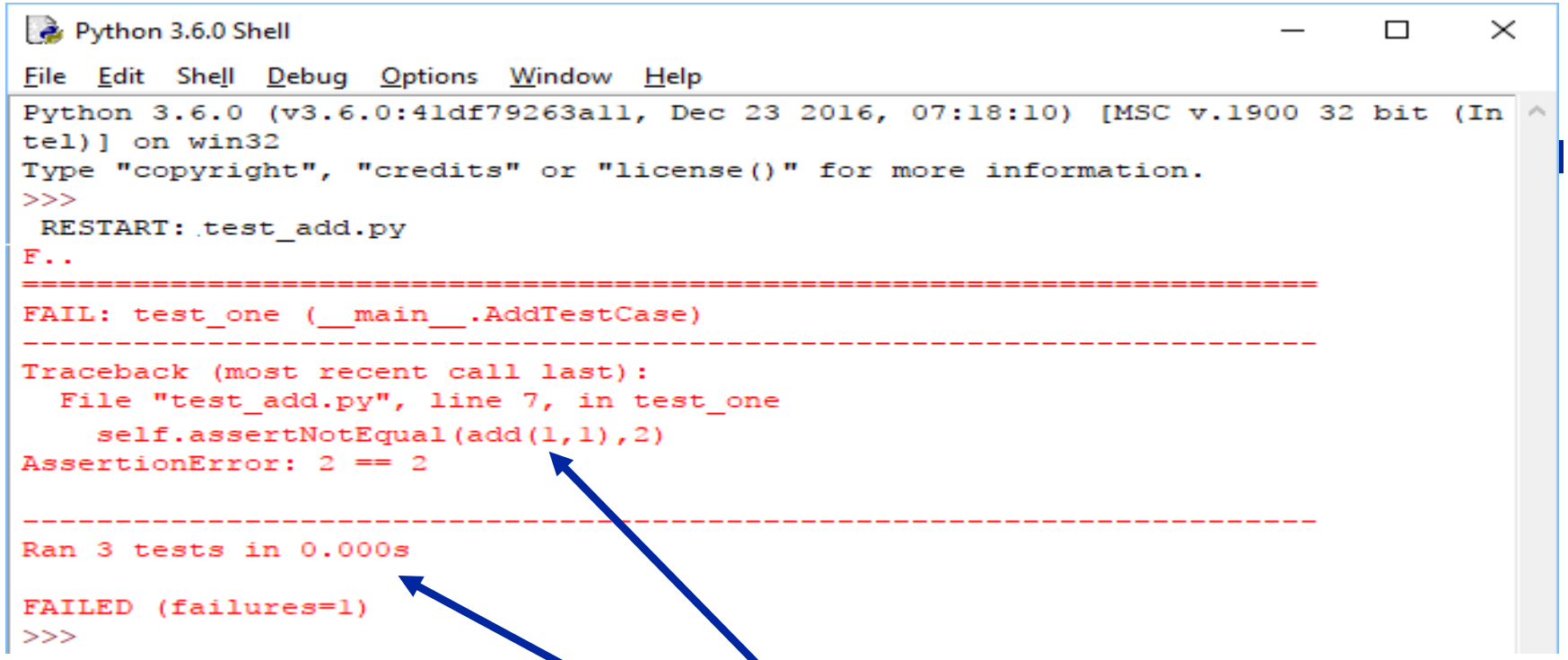
Broken test case that expects the return value from add(1,1) not to equal 2

Positive test case: Check if the return value from add(1,2) is 3

Negative test case: Check if the return value from add(1,2) is not equal to 5

Allows us to run our tests as if it was the main line of the program (i.e. the place the program starts)

(2) What happens when we run this broken test case?



In IDLE, you can use the F5 shortcut key to automatically run the code.

All three tests ran, there has been one “Failure”, in line 7

(3) Python code: is_prime.py

```
def is_prime(number):
#funtion goes here
    for element in range(2,number):
        if number % element == 0:
            print(number)
            print("NOT PRIME")
            return False
    print(number)
    print("PRIME")
    return True

x=int(input("enter:"))
is_prime(x)
```

(3) Python code: corresponding unit test code

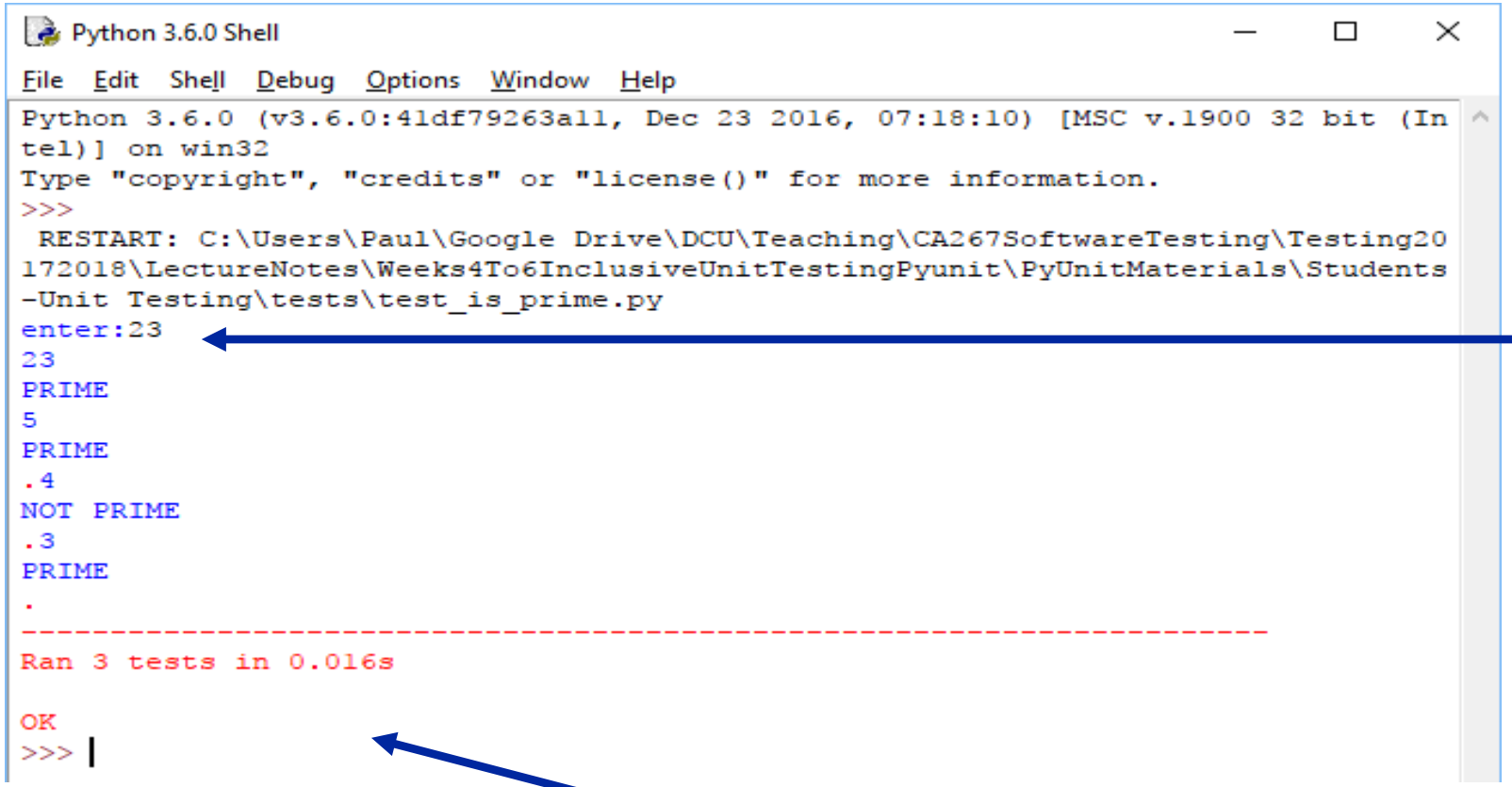
```
import unittest
from code.primes import is_prime

class PrimesTestCase(unittest.TestCase):

    def test_is_five_prime(self):
        self.assertTrue(is_prime(5))

    def test_is_four_prime(self):
        self.assertFalse(is_prime(4))

    def test_is_three_prime(self):
        self.assertTrue(is_prime(3))


if __name__ == '__main__':
    unittest.main()
```

Check that the return value from is_prime() is true for 5

Check that the return value from is_prime() is false for 4

Check that the return value from is_prime() is true for 3

(3) What happens when we run this code?



Manually entered “23” at the command line. Note that this is not part of the pyunit tests.

All three tests ran, everything is “OK”

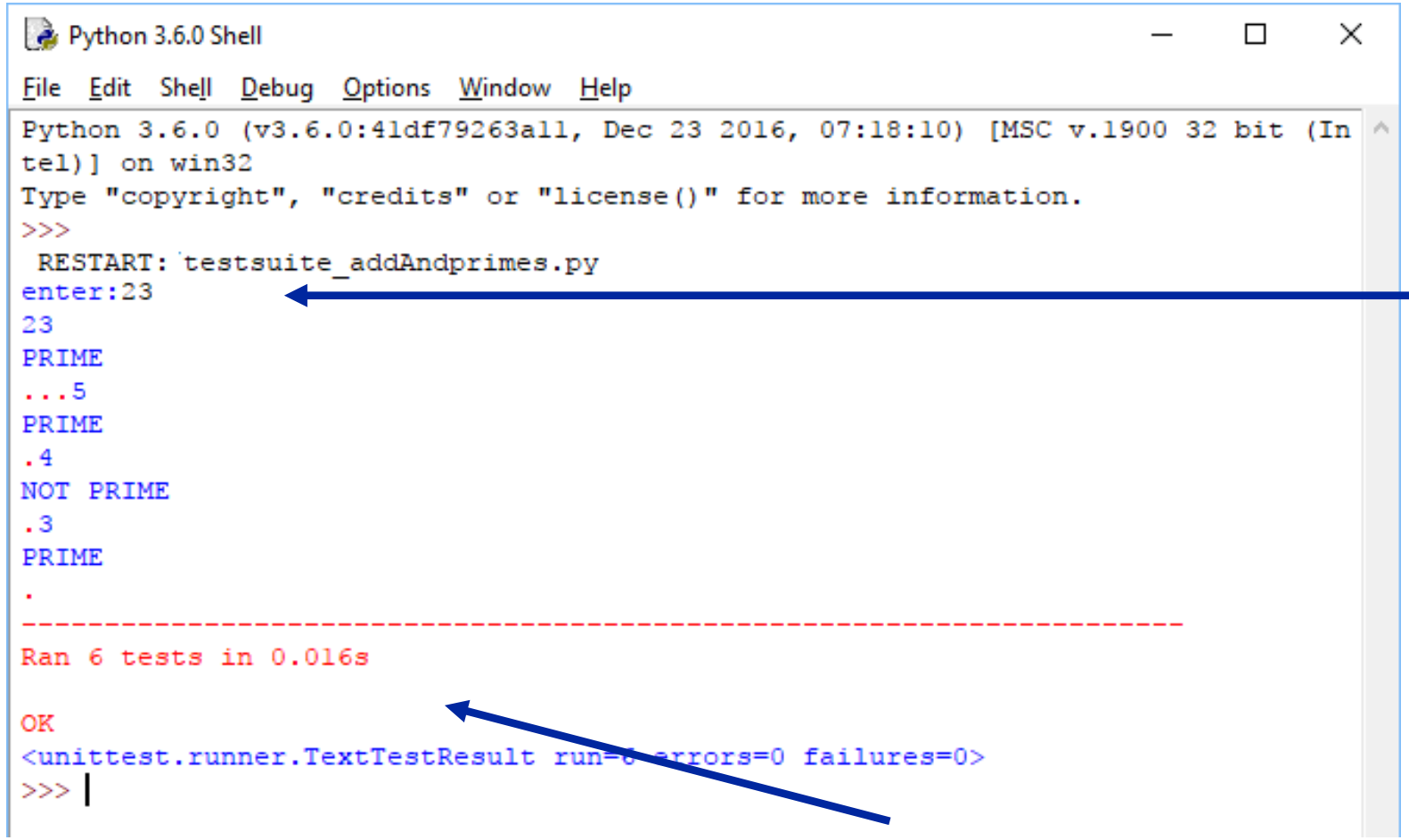
(4) Python test suite code: add and primes

```
import unittest

from test_add import AddTestCase
from test_is_prime import PrimesTestCase


def my_suite():
    suite = unittest.TestSuite() #instance of Testsuite
    suite.addTest(unittest.makeSuite(AddTestCase))# addTest is a keyword
    suite.addTest(unittest.makeSuite(PrimesTestCase))
    runner = unittest.TextTestRunner()
    print(runner.run(suite)) # run the suite and print out the results

my_suite()
```

Import ‘AddTestCase’ and ‘’PrimesTestCase’

Create my test suite

(4) What happens when we run this code?



Manually entered “23” at the command line. Note that this is not part of the pyunit tests.

Note: both sets of test cases and source files are correct prior to running this test suite.

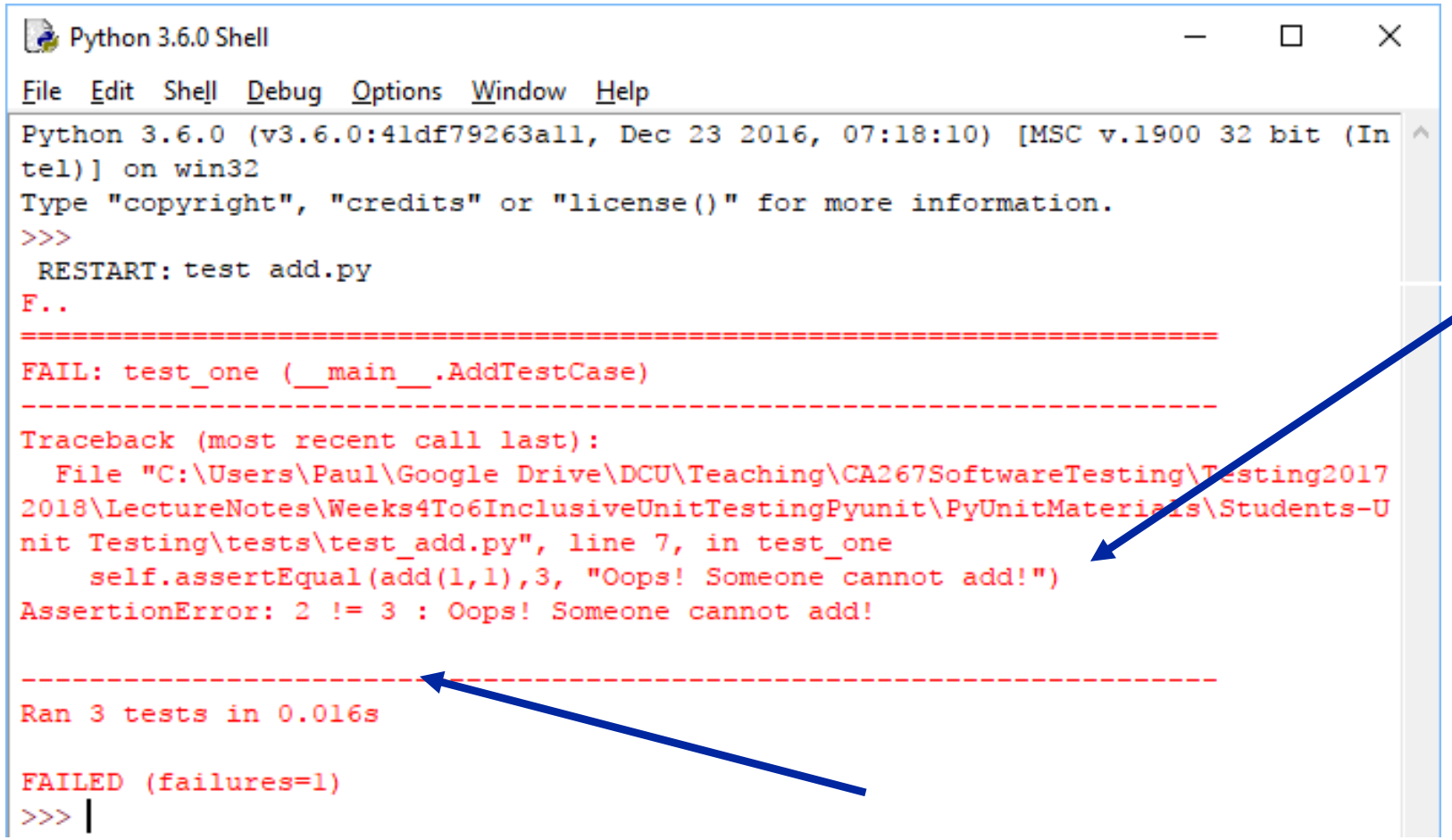
All 6 tests ran, everything is “OK”

Example: Adding a custom message to a unit test

```
import unittest
from code.add import add

class AddTestCase(unittest.TestCase):

    def test_one(self):
        self.assertEqual(add(1,1),3, "Oops! Someone cannot add!")

    def test_two(self):
        self.assertEqual(add(1,2),3)

    def test_three(self):
        self.assertNotEqual(add(1,3),5)

if __name__ == '__main__':
    unittest.main()
```

Custom message. In practice, a more informative / professional message should be added!

What happens when we run this code?



Custom message appears in output.

All 3 tests ran, but one failed.

PyUnit for large scale automated testing?

We have looked at some very small-scale examples constructed just for the purpose of learning to use pyunit at an introductory level.

These same principles can be scaled to allow great numbers of test cases to be executed automatically using pyunit.

This is a powerful way to continually perform tests, especially when used in conjunction with continuous integration

PyUnit for integration and system testing?

Since the unit test framework can invoke any code and in any sequence, it can be considered to be quite effective for integration testing (and possibly some system testing also).

We can therefore orchestrate entire automated integration testing sweeps that can run continuously

Together with unit testing, this approach is important in enabling continuous software engineering

But to truly continuous software engineer, other things are also needed (consider GUI testing, automated deployment, monitoring and control, etc…)